

Astralis Core Fiscal

API para cadastro de usuarios e empresas, autenticao e processamento de arquivos fiscais XML.

O projeto esta em desenvolvimento e utiliza Spring Boot com uma organizacao inspirada em Clean Architecture, separando dominio, casos de uso, adaptadores e infraestrutura.

Status atual

Ja implementado:

- Cadastro e login de usuarios.
- Autenticacao stateless com JWT.
- Criptografia de senhas com BCrypt.
- Busca e exclusao de usuarios.
- Cadastro, atualizacao, consulta e exclusao de empresas.
- Upload de arquivos ZIP contendo XMLs fiscais.
- Extracao dos XMLs e soma dos valores de PIS e COFINS.
- Persistencia com Spring Data JPA e MySQL.
- Perfil H2 para testes.
- Tratamento global de excecoes.
- Configuracoes para desenvolvimento, testes e producao.
- Testes unitarios e de integracao basicos.

Ainda pendente:

- Implementar o calculo completo de ICMS.
- Criar o fluxo completo de apuracao fiscal.
- Persistir e consultar relatorios por empresa e competencia.
- Expor endpoints de relatorios fiscais.
- Completar validacoes de XML, arquivos e CNPJ.
- Implementar autorizacao por usuario, perfil e empresa.
- Documentar a API com OpenAPI/Swagger.
- Ampliar os testes de controllers, seguranca e processamento XML.
- Remover credenciais e segredos fixos das configuracoes de desenvolvimento.
- Criar Docker, CI/CD e monitoramento.

Tecnologias

- Java 17
- Spring Boot 3.5.6
- Spring Web

- Spring Data JPA
- Spring Security
- JWT com JJWT 0.12.5
- MySQL
- H2 para testes
- Maven
- Lombok
- Bean Validation

Arquitetura

```
src/main/java/com/astralisCaerulis/AstralisCoreFiscal/
■■■ Core/
■   ■■■ domain/           # Modelos, enums e contratos do dominio
■   ■■■ Exceptions/       # Excecoes de negocio
■■■ Application/
■   ■■■ dtos/             # Objetos de entrada e saida
■   ■■■ ports/            # Portas da aplicacao
■   ■■■ useCases/         # Regras de cada caso de uso
■■■ adapters/
■   ■■■ persistence/      # Entidades, mappers e repositórios JPA
■   ■■■ web/              # Controllers, handlers e utilitários HTTP
■■■ infrastructure/
    ■■■ config/           # Configuracoes do Spring
    ■■■ security/         # JWT, filtros e handlers de seguranca
```

Casos de uso existentes

Usuarios

- CreateUserCase
- LoginUserCase
- UpdateUserUseCase
- DeleteUserCase
- FindUserByIdUseCase
- FindUserByEmailUseCase

Empresas

- CreateEnterpriseCase
- UpdateEnterpriseCase
- DeleteEnterpriseCase
- FindEnterpriseByIdUseCase
- FindEnterpriseByCnpjUseCase

Relatorios fiscais

- ProcessFiscalRepostUseCase

O processamento atual lê as tags vPIS e vCOFINS dos XMLs e retorna a soma dos dois valores. O modelo TaxApurationReport já existe, mas o fluxo completo de apuração ainda precisa ser implementado.

API

O contexto-base da aplicação é:

/astralis-fiscal/v1

Usuarios

Metodo	Endpoint	Autenticacao
POST	/users/create	Nao
POST	/users/login	Nao
GET	/users/by-id/{id}	Sim
GET	/users/by-email/{email}	Sim
DELETE	/users/delete/{id}	Sim

Empresas

Metodo	Endpoint	Autenticacao
POST	/enterprises/create	Sim
PUT	/enterprises/update/{id}	Sim
DELETE	/enterprises/delete/{id}	Sim
GET	/enterprises/by-id/{id}	Sim
GET	/enterprises/by-cnpj/{cnpj}	Sim

Upload fiscal

Metodo	Endpoint	Autenticacao
POST	/fiscal-report/upload-zip	Atualmente nao

O endpoint recebe um arquivo multipart/form-data no campo file.

```
curl -X POST \
  http://localhost:8080/astralis-fiscal/v1/fiscal-report/upload-zip \
  -F "file=@notas-fiscais.zip"
```

O ZIP deve conter arquivos XML. A resposta atual possui o formato:

Total de PIS + COFINS: 123.45

Configuracao

Requisitos:

- Java 17 ou superior.
- MySQL para desenvolvimento.
- Maven Wrapper incluído no projeto.

O perfil de desenvolvimento usa o banco `astralis_fiscal_dev`. Antes de executar a aplicação, configure a conexão com o banco e os valores de JWT em `application-dev.properties` ou por variáveis de ambiente.

O perfil de produção espera:

- `DATABASE_URL`
- `DATABASE_USERNAME`
- `DATABASE_PASSWORD`
- `JWT_SECRET`
- `JWT_EXPIRATION` opcional
- `SERVER_PORT` opcional

Nunca publique senhas ou segredos JWT no repositório. O segredo atualmente usado em desenvolvimento deve ser substituído por uma configuração local ou variável de ambiente.

Como executar

Linux ou macOS:

```
./mvnw spring-boot:run
```

Windows:

```
mvnw.cmd spring-boot:run
```

Com a configuração padrão, a API fica disponível em:

```
http://localhost:8080/astralis-fiscal/v1
```

Testes

```
./mvnw test
```

Os testes existentes cobrem criação e login de usuários, criação de empresas, handlers de segurança, tratamento de exceções e inicialização da aplicação.

Próximos passos recomendados

1. Proteger o upload fiscal com autenticação e validar tamanho e tipo do arquivo.
2. Implementar o cálculo de ICMS e o caso de uso de apuração fiscal.
3. Persistir relatórios por empresa e competência.
4. Criar endpoints de consulta de apurações.
5. Remover segredos do controle de versão.
6. Criar testes para o processamento de ZIP/XML e para os controllers.
7. Adicionar Swagger, Docker Compose e pipeline de CI.